

Example: Creating a Custom Resource Filter

1. Create CustomResourceFilter.cs

This filter logs the request start time and can short-circuit the request to return a response before reaching the action method.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.Extensions.Logging;
using System.Diagnostics;

public class CustomResourceFilter : IResourceFilter
{
    private readonly ILogger<CustomResourceFilter> _logger;
    private Stopwatch _stopwatch;

    public CustomResourceFilter(ILogger<CustomResourceFilter> logger)
    {
        _logger = logger;
    }

    public void OnResourceExecuting(ResourceExecutingContext context)
    {
        _stopwatch = Stopwatch.StartNew();
        _logger.LogInformation("Resource Filter: Request processing
started.");

        // Short-circuit the request (Uncomment if needed)
        // context.Result = new ContentResult() { Content = "Request was
blocked by Resource Filter." };
    }

    public void OnResourceExecuted(ResourceExecutedContext context)
    {
        _stopwatch.Stop();
        _logger.LogInformation($"Resource Filter: Request processing completed
in {_stopwatch.ElapsedMilliseconds} ms.");
    }
}
```

2. Register the Filter Globally

In `Program.cs`, add the filter globally so it applies to all requests:

```
builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add<CustomResourceFilter>(); // Registering the resource
filter globally
});
```

3. Apply to a Controller or Action (Optional)

If you don't want it globally, apply it to a specific controller or action:

```
public class HomeController : Controller
{
    [CustomResourceFilter] // Applying the filter to a single action
    public IActionResult Index()
    {
        return View();
    }
}
```

4. Output (Console Logs)

When a request is made to the action:

```
info: CustomResourceFilter[0]
      Resource Filter: Request processing started.
info: CustomResourceFilter[0]
      Resource Filter: Request processing completed in 10 ms.
```

Key Takeaways

- Resource Filters execute before and after the request pipeline (excluding authentication/authorization).
- They are ideal for caching, short-circuiting, and modifying requests.
- They help improve performance by preventing unnecessary request processing.

Issue: Constructor Injection in Attribute-Based Filter

The error occurs because `[ResourceFilter]` is applied as an attribute, but your `ResourceFilter` class has a constructor dependency on `ILogger<ResourceFilter>`.

 Attributes must have parameterless constructors, but your filter requires `ILogger<ResourceFilter>`, which cannot be provided at compile-time.

Solution: Use `ServiceFilter` or `TypeFilter`

Since `ResourceFilter` depends on `ILogger<ResourceFilter>`, we must let **ASP.NET Core Dependency Injection (DI)** handle it.

♦ Solution 1: Use `[ServiceFilter]` (Preferred)

1. Register the Filter in `Program.cs`

Modify `Program.cs` to register `ResourceFilter`:

```
builder.Services.AddScoped<ResourceFilter>();  
// Registering the filter as a service
```

2. Use `[ServiceFilter]` Instead of `[ResourceFilter]`

Modify `HomeController.cs`:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.Extensions.Logging;  
using CoreDemo.Filters;  
using CoreDemo.Models;  
using System.Diagnostics;
```

```
namespace CoreDemo.Controllers
```

```
{  
    public class HomeController : Controller  
    {  
        private readonly ILogger<HomeController> _logger;  
  
        public HomeController(ILogger<HomeController> logger)  
        {  
            _logger = logger;  
        }  
  
        public IActionResult Index()  
        {  
            return View();  
        }  
    }  
}
```

`[ServiceFilter(typeof(ResourceFilter))]` //  Correct way to apply a filter with DI

```
public IActionResult Privacy()  
{  
    return View();  
}
```

```

    }

    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None,
NoStore = true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id
?? HttpContext.TraceIdentifier });
    }
}

```

✅ Now the filter will be resolved using DI, and it will work correctly!

♦ **Solution 2: Use [TypeFilter]**

If you don't want to register the filter in `Program.cs`, use `[TypeFilter]` instead:

```

[TypeFilter(typeof(ResourceFilter))] // ✅ Works without registering in
Program.cs
public IActionResult Privacy()
{
    return View();
}

```

Difference Between [ServiceFilter] and [TypeFilter]

Feature	ServiceFilter	TypeFilter
Requires DI registration	✓ Yes (<code>Program.cs</code>)	✗ No registration needed
Supports constructor injection	✓ Yes	✓ Yes
When to use	If filter is registered as a service	If filter is not registered as a service

Key Takeaways

1. Do NOT use [ResourceFilter] directly when constructor injection is needed.
2. Use [ServiceFilter] if the filter is registered in `Program.cs`.
3. Use [TypeFilter] if you don't want to register the filter in `Program.cs`.
4. Ensure DI can resolve dependencies like `ILogger<ResourceFilter>`.

Yes, if you define your `ResourceFilter` as an **attribute** (`Attribute`, `IResourceFilter`), then **you cannot use constructor injection** for dependencies like `ILogger<ResourceFilter>`.

🔴 Problem with Using `Attribute`, `IResourceFilter`

When you apply `[ResourceFilter]` directly:

```
public class ResourceFilter : Attribute, IResourceFilter
{
    private readonly ILogger<ResourceFilter> _logger; // ❌ Cannot inject this

    public ResourceFilter(ILogger<ResourceFilter> logger) // ❌ Constructor injection won't work
    {
        _logger = logger;
    }

    public void OnResourceExecuting(ResourceExecutingContext context)
    {
        _logger.LogInformation("Resource Filter - OnResourceExecuting");
    }

    public void OnResourceExecuted(ResourceExecutedContext context)
    {
        _logger.LogInformation("Resource Filter - OnResourceExecuted");
    }
}
```

🔥 **This won't work** because attributes **must have parameterless constructors**. ASP.NET Core **cannot inject dependencies into attributes** at runtime.

✅ Solution: Use `ServiceFilter` or `TypeFilter`

If your filter needs dependencies (like `ILogger`), you **must remove `Attribute`** from your class and use **either `ServiceFilter` or `TypeFilter`**.

1️⃣ Option 1: Use `ServiceFilter` (Preferred)

1. **Remove `Attribute`** from your filter:

```
csharp
CopyEdit
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.Extensions.Logging;
using System.Diagnostics;

public class ResourceFilter : IResourceFilter
{
    private readonly ILogger<ResourceFilter> _logger;
    private Stopwatch _stopwatch;

    public ResourceFilter(ILogger<ResourceFilter> logger)
    {
        _logger = logger;
    }
}
```

```

public void OnResourceExecuting(ResourceExecutingContext context)
{
    _stopwatch = Stopwatch.StartNew();
    _logger.LogInformation("Resource Filter - OnResourceExecuting");
}

public void OnResourceExecuted(ResourceExecutedContext context)
{
    _stopwatch.Stop();
    _logger.LogInformation($"Resource Filter - OnResourceExecuted in { _stopwatch.ElapsedMilliseconds} ms");
}
}

```

2. Register the Filter in Program.cs:

```

csharp
CopyEdit
builder.Services.AddScoped<ResourceFilter>(); // Register the filter as a
service

```

3. Apply it using [ServiceFilter]:

```

csharp
CopyEdit
[ServiceFilter(typeof(ResourceFilter))] // ✅ This works with DI
public IActionResult Privacy()
{
    return View();
}

```

2] Option 2: Use TypeFilter (No DI Registration Needed)

If you don't want to register the filter in Program.cs, use [TypeFilter]:

```

csharp
CopyEdit
[TypeFilter(typeof(ResourceFilter))] // ✅ Works without registering in
Program.cs
public IActionResult Privacy()
{
    return View();
}

```

📌 Key Takeaways

1. You cannot inject dependencies (**ILogger**) in an attribute-based filter (**Attribute, IResourceFilter**).
2. Remove **Attribute** from your filter if you need dependency injection.
3. Use **[ServiceFilter]** if the filter is registered in Program.cs.
4. Use **[TypeFilter]** if you don't want to register it in Program.cs.